

LPII - Mapeamento da Sintaxe de Python para Java

Em LPII, será desenvolvida uma aplicação desktop na linguagem Java, que é uma linguagem orientada a objetos bastante utilizada. Neste tutorial serão comentadas as diferenças entre código Python e Java, ilustrando com a conversão de um projeto em Python para um projeto em Java. Em ambas as aplicações, todo o código será representado em subdiretórios do diretório [src](#).

O projeto de ilustração suporta reservas de obras que podem ser livros ou trabalhos de pós graduação em uma biblioteca universitária, na qual os consulentes (usuários da biblioteca) podem ser: alunos, professores ou técnicos administrativos.

Instale a seguinte versão do Java (jdk-17_windows-x64_bin):

- site: <https://www.oracle.com/java/technologies/javase/jdk17-archive-downloads.html>
 - Windows x64 Installer

Instale a seguinte versão do IDE (Integrated Development Environment) NetBeans (Apache NetBeans 12.6) para editar, compilar e executar seus desenvolvimentos em Java:

- Apache-NetBeans-12.6-bin-windows-x64.exe

Em cada uma das seções seguintes serão ilustradas: (a) a implementação em Python; (b) a implementação correspondente em Java; e (c) a descrição do mapeamento de cada novo conceito em Python para o seu correspondente em Java. Nas três seções iniciais serão ilustradas as entidades do projeto: (a) a classe Consulente na seção 1; (b) a superclasse Obra e suas subclasses Livro e TrabalhoPósGraduação, na seção 2; e (c) a classe Reserva, que referencia um objeto da classe Consulente e um objeto da classe Obra, na seção 3. A classe de controle Biblioteca será ilustrada na seção 4. E finalmente, será representada a saída do projeto na seção 5.

1 – Implementando uma entidade simples

Nesta seção, será ilustrada a implementação da classe Consulente. Pelo fato do tipo do consulente (aluno, professor ou técnico administrativo) ser o único atributo que diferencia os dados dos três tipos de consulentes, não é necessário utilizar o conceito de herança, bastando incluir o atributo tipo na classe Consulente.

A seguir, a implementação da classe [Consulente](#) em Python, no módulo [consulente](#) do diretório [entidades](#).

```
consulentes = {}

def get_consulentes(): return consulentes

def inserir_consulente(consulente):
    cpf = consulente.cpf
    if cpf not in consulentes.keys():
        consulentes[cpf] = consulente
    else: print('Erro -> consulente já cadastrado com cpf : ' + cpf)
```

```
class Consulente:

    def __init__(self, nome, cpf, email, tipo):
        self.nome = nome
        self.cpf = cpf
        self.email = email
        self.tipo = tipo if tipo in ['aluno', 'técnico administrativo', 'professor']
            else 'indefinido'

    def __str__(self):
        return 'Consulente (' + self.tipo + '): ' + self.nome + ' - CPF: ' + self.cpf
            + ' - email: ' + self.email
```

A seguir, a implementação da classe **Consulente** em Java, no arquivo **Consulente** do pacote **entidades**. Em Java, cada classe é representada em um arquivo separado, com o mesmo nome da classe. Diretório em Java é denominado pacote, representado pela palavra reservada **package**.

```
package entidades;

import java.util.HashMap;

public class Consulente {

    private static HashMap<String, Consulente> consulentes = new HashMap();
    public static HashMap<String, Consulente> getConsulentes() { return consulentes; }

    public static void inserir(Consulente consulente) {
        String cpf = consulente.getCpf();
        if (consulentes.get(cpf) == null) consulentes.put(cpf, consulente);
        else System.out.println("Erro -> consulente já cadastrado com cpf : " + cpf);
    }

    public enum Tipo {aluno, técnico_administrativo, professor};

    private String nome, cpf, email;
    private Tipo tipo;

    public Consulente(String nome, String cpf, String email, Tipo tipo) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
        this.tipo = tipo;
    }

    public String getNome() { return nome; }
    public String getCpf() { return cpf; }
    public String getEmail() { return email; }
    public Tipo getTipo() { return tipo; }

    public String toString() {
        return "Consulente (" + tipo + "): " + nome + " - CPF : " + cpf + " - email : " + email;
    }

}
```

A sintaxe de Java foi definida a partir da linguagem C++, que por sua vez foi definida a partir da linguagem C. Portanto, a forma de representar blocos de comandos, utilizar tipos para declarar variáveis e parâmetros de funções, representação de condições de comandos condicionais e de repetição serão familiares para quem já programou em C.

Antecedendo à definição da classe são declarados: (a) nome do pacote (**package**), que corresponde ao diretório no qual o arquivo da classe é definido; e (b) as importações (**import**) de classes definidas em outros pacotes. Classes definidas no mesmo pacote não precisam ser importadas. Na importação de cada classe, é especificada a hierarquia de pacotes e o nome da classe importada. Por exemplo, a importação `import java.util.HashMap`, indica que a classe importada `HashMap` é definida no pacote `util`, que por sua vez é definido no pacote `java`. Java disponibiliza uma grande variedade de pacotes e funções que poderão ser importados e utilizados para desenvolver aplicações.

O pacote `java.lang` é o pacote mais básico de Java. Por ser muito utilizado, é o único pacote cujas classes não precisam ser importadas por classes de outros pacotes. `String` é uma de suas classes. Observe que a importação `java.lang.String` não é necessária.

Em Python, declarações de atribuição (ex: `nome = "Ana Julia";`) ou de comando (ex: `if` e `for`) internos a um determinado escopo (bloco) são indentados (com Tab) para caracterizar que pertencem a esse bloco. Em Java blocos são delimitados pelos caracteres `{ }`. A indentação continua a ser utilizada por questões de legibilidade, mas o compilador Java se guia pelos delimitadores `{ }`, para definir quais comandos são internos a um determinado bloco.

Observe da definição da classe `Consulente`, que o bloco que abrange todos as variáveis e funções da classe, bem como os blocos que contém os comandos das funções também são delimitados pelos caracteres `{ }`.

Diferentemente de Python, as declarações de pacote, importações, variáveis e comandos são finalizados pelo caracter `;`.

Todos os dados e funções são representadas dentro de uma classe. Variáveis que recebem valores específicos para cada objeto da classe são consideradas dinâmicas, porque o construtor da classe é chamado com diferentes valores de argumentos para os parâmetros que utiliza para inicializar as variáveis de cada novo objeto criado. Variáveis e funções globais são consideradas estáticas, caracterizadas pela palavra reservada `static`, porque seu valores são únicos para todo o projeto.

Python é uma linguagem com tipagem dinâmica, o que significa que o tipo de uma variável é verificado somente em tempo de execução. Neste caso, a definição de uma variável não envolve a definição do seu tipo e, além disso, é possível atribuir valores de diferentes tipos para uma dada variável, ao longo do código. No entanto, Java é uma linguagem com tipagem estática, o que significa que o tipo de uma dada variável é checado em tempo de compilação. Neste caso, é necessário definir o tipo da variável, para a qual somente poderão ser atribuídos valores do mesmo tipo.

Em Java é possível definir um tipo enumerado (`enum`), para o qual é possível enumerar os valores que sua variável poderá receber:

```
enum Tipo {aluno, técnico_administrativo, professor}; // definição do Tipo
Tipo tipo = aluno; // definição e atribuição da variável baseada no enumerado
```

Python não suporta tipo enumerados, e neste caso a alternativa é utilizar strings e checar no construtor se o parâmetro recebido pertence a um determinado conjunto de valores:

```
self.tipo = tipo if tipo in ['aluno', 'técnico administrativo', 'professor'] else 'indefinido'
```

O fato do tipo do enumerado ilustrado se chamar **Tipo** é uma mera coincidência. Se fosse para enumerar valores de estados civis, o tipo enumerado seria denominado **EstadoCivil**:

```
enum EstadoCivil {solteiro, casado, separado, viúvo};
```

Em Java, todas as declarações de variáveis, parâmetros e tipos de retorno de funções são precedidas pelo tipo da variável, do parâmetro ou do retorno. Veja abaixo ilustrações de usos de tipos (demarcados em negrito) para definir: enumerado, variáveis de objeto, parâmetros de métodos (construtor **Consulente** e **setNome**) e tipo de retorno para métodos (**getNome** e **setNome**). Nesses exemplos, foram omitidas as palavras reservadas **public** e **private**, cujo significado e utilização serão explicados oportunamente.

```
enum Tipo {aluno, técnico_administrativo, professor}; // tipo enumerado
String nome, cpf, email; // declarações de variáveis com um mesmo tipo
Tipo tipo; // variável definida a partir de um tipo enumerado
Consulente(String nome, String cpf, String email, Tipo tipo) { ... } // tipo de parâmetros
String getNome() { return nome; } // tipo de retorno do método
void setNome(String nome) { this.nome = nome; } // tipo de retorno e tipo de parâmetro
```

O construtor é denominado pelo próprio nome da classe, neste caso **Consulente**. A palavra reservada **this** (este) é utilizada para diferenciar variáveis de objeto de parâmetros de mesmo nome.

```
Consulente(String nome, String cpf, String email, Tipo tipo) {
    this.nome = nome; // o primeiro é a variável nome e o segundo é o parâmetro nome
    ...
}
```

O construtor é o único método para o qual não é necessário definir o tipo de retorno do método. Dado que o construtor é utilizado para criar um objeto da classe, o seu tipo de retorno seria a própria classe. Como o tipo de retorno e o nome do método teriam o próprio nome da classe, é omitido o tipo de retorno para evitar a duplicação. O código abaixo é incorreto, e esta sendo mostrado somente para mostrar a duplicação que ocorreria, caso fosse utilizado.

```
Consulente Consulente(String nome, String cpf, String email, Tipo tipo) { ... }
```

Nos demais métodos, quando não existe tipo de retorno, é utilizada a palavra reservada **void** (vazio) para designar que o método não retorna nenhum valor. O método **setNome** não retorna nenhum valor, e foi ilustrado acima para com utilização do tipo de retorno **void**. Esse método não é utilizado neste projeto, mas é definido para alterar o valor da variável de um objeto.

Variáveis do tipo **String** em Java são delimitadas pelos caracteres **" "**. Variáveis do tipo **char** (caracter) são delimitadas pelos caracteres **' '**. Como por exemplo:

```
String nome = "Ana Julia";
char letra = 'a';
```

No dois exemplos acima, a declaração e a atribuição foram implementadas em um único comando, mas obviamente você pode separá-las. E neste caso a declaração da variável é feita uma única vez, mas a variável pode receber mais de uma atribuição, desde que seja com valores do tipo que foi utilizado para declarar a variável. Veja o exemplo:

```
String nome;
nome = "Ana Julia";
nome = "Adriana";
```

É muito comum em Java, a utilização de modificadores de acesso. O modificar de acesso **public** (público) quando utilizado com uma classe ou um tipo enumerado, indica que eles são visíveis em classes de outros pacotes e, portanto, podem ser importados por essas classes. O modificador de acesso **public** quando utilizado com funções e com métodos indica que eles podem ser chamados fora da classe na qual foram definidos. O modificador de acesso **private** (privado) é normalmente utilizado para denotar que variáveis ou métodos não são acessíveis fora da classe na qual foram definidos.

Classes são definidas como públicas, para que seus objetos, tipos enumerados, funções e métodos possam ser utilizados por classes de outros pacotes. Nestes casos, é necessário importar somente a classe e o seus enumerados, como será mostrado na seção 3 na ilustração da classe **Reserva**, na qual serão importados a classe **Consulente** e o enumerado **Consulente.Tipo**. Portanto, não é necessário importar as funções globais de uma determinada classe, para utilizá-las em classes de outros pacotes.

Funções globais (**static**) e tipos enumerados (**enum**) são sempre públicos, para que possam ser utilizados por classes de outros pacotes. Métodos podem ser públicos ou privados. Somente os métodos públicos podem ser chamados em outras classes, a partir de um objeto da classe na qual foram definidos. Métodos privados só podem ser utilizados na classe na qual foram definidos. Variáveis em geral são privadas, o que significa que só poderão ser lidas a partir de métodos públicos de leitura (ex: **getNome**) e alteradas a partir de métodos públicos de alteração (ex: **setNome**).

Em Python quando queremos armazenar um objeto atrelado à sua chave para posterior recuperação, utilizamos a estrutura de dados dicionário. Em Java podemos obter o mesmo efeito utilizando a classe **HashMap**, definida a partir de dois tipos: o tipo da chave e o tipo do objeto (ex: **HashMap<String, Consulente>**). Como a variável é global e privada deverá ser declarada com **private** e **static**. No exemplo abaixo o primeiro comando declara a variável global **consulentes** e inicializa com um objeto da classe **HashMap**. A criação de um objeto é feita com a utilização da palavra reservada **new** (novo) para chamar o construtor da classe (**HashMap**). Pelo fato da variável global ser declarada com privada, o seu acesso será realizado a partir do método público e estático (global) **getConsulentes**, cujo tipo de retorno é o mesmo tipo da variável declarada.

```
private static HashMap<String, Consulente> consulentes = new HashMap();
public static HashMap<String, Consulente> getConsulentes() { return consulentes; }
```

Na função pública e estática **inserir** é ilustrado como obter um objeto a partir de sua chave (**consulentes.get(cpf)**) e como inserir o objeto **consulente**, atrelado à sua chave **cpf**, no objeto **consulentes** da classe **HashMap<String, Consulente>** (**consulentes.put(cpf, consulente)**). O atributo **cpf** do objeto **consulente** é acessado com a utilização do método público **getCpf**. A condição utilizada no comando **if** é encapsulada pelos caracteres **()**. Em Java, nulo é representado pela palavra reservada **null**.

```
public static void inserir(Consulente consulente) {
    String cpf = consulente.getCpf();
    if (consulentes.get(cpf) == null) consulentes.put(cpf, consulente);
    else System.out.println("Erro -> consulente já cadastrado com cpf : " + cpf);
}
```

Na função `inserir` é chamado o método `println` para imprimir o erro. Para imprimir em Python basta chamar a função `print`, mas em Java é necessário utilizar `System.out.println`:

```
System.out.println("Erro -> consulente já cadastrado com cpf : " + cpf);
```

Para gerar um string que representa os atributos do um objeto é definido o método `String toString()`. Para todas as classes esse método tem a mesma assinatura, ou seja: o mesmo nome (`toString`), o mesmo tipo de retorno (`String`) e uma lista de parâmetros vazia.

Nos métodos não é necessário utilizar `this` para acessar as variáveis do objeto. Também não é necessário converter variáveis para string para utilizar o concatenador de strings `+`. Ao contrário, o valor de qualquer tipo de variável concatenada com o operador `+` será convertido para string para realizar a concatenação, mesmo que seja um objeto.

2 – Implementando uma superclasse e suas subclasses

Nesta seção, será ilustrada a implementação da superclasse `Obra` e de suas subclasses. A classe `Data` também será ilustrada, dado que é utilizada pelas subclasse `TrabalhoPósGraduação`.

A classe `Data` em Python é implementada no módulo `data` do diretório `util`.

```
from datetime import date

class Data:

    def __init__(self, dia, mês, ano):
        self.dia = dia
        self.mês = mês
        self.ano = ano

    def __str__(self):
        if self.dia < 10: data_str = '0' + str(self.dia)
        else: data_str = str(self.dia)
        if self.mês < 10: data_str += "/0" + str(self.mês) + "/"
        else: data_str += "/" + str(self.mês) + "/";
        data_str += str(self.ano)
        return data_str

    def __eq__(self, data):
        if str(self) == str(data): return True
        return False

    def __ne__(self, data): return not self == data

    def __gt__(self, data):
        if self.ano > data.ano: return True
        elif self.ano < data.ano: return False
        if self.mês > data.mês: return True
        elif self.mês < data.mês: return False
        if self.dia > data.dia: return True
        elif self.ano < data.ano: return False
        return False

    def __lt__(self, data):
        if self.ano < data.ano: return True
        elif self.ano > data.ano: return False
        if self.mês < data.mês: return True
        elif self.mês > data.mês: return False
        if self.dia < data.dia: return True
        elif self.ano > data.ano: return False
        return False
```

```
def __ge__(self, data):
    if self < data: return False
    else: return True

def __le__(self, data):
    if self > data: return False
    else: return True

def calcular_idade(self):
    dia_atual_str, mês_atual_str, ano_atual_str = date.today().strftime("%d/%m/%Y").split('/')
    dia_atual, mês_atual, ano_atual = int(dia_atual_str), int(mês_atual_str), int(ano_atual_str)
    idade = ano_atual - self.ano
    if mês_atual < self.mês or (mês_atual == self.mês and dia_atual < self.dia): idade -= 1
    return idade
```

A classe **Data** em Java é implementada no arquivo **Data** do pacote **util**.

```
package util;

import java.util.Calendar;
import java.util.GregorianCalendar;

public class Data {

    private int dia, mês, ano;

    public Data(int dia, int mês, int ano) {
        this.dia = dia;
        this.mês = mês;
        this.ano = ano;
    }

    public int getDia() { return dia; }
    public int getMês() { return mês; }
    public int getAno() { return ano; }

    public int calcularIdade () {
        GregorianCalendar data_atualGregorianCalendar = new GregorianCalendar();
        int dia_atual = data_atualGregorianCalendar.get(Calendar.DAY_OF_MONTH);
        int mês_atual = data_atualGregorianCalendar.get(Calendar.MONTH) + 1;
        // GregorianCalendar: mês varia de 0 a 11
        int ano_atual = data_atualGregorianCalendar.get(Calendar.YEAR);
        int idade = ano_atual - ano;
        if ((mês_atual < mês) || ((mês_atual == mês) && (dia_atual < dia))) idade--;
        return idade;
    }

    public int compareTo(Data data) {
        if (ano > data.getAno()) return 1;
        if (ano < data.getAno()) return -1;
        if (mês > data.getMês()) return 1;
        if (mês < data.getMês()) return -1;
        if (dia > data.getDia()) return 1;
        if (dia < data.getDia()) return -1;
        return 0;
    }

    public String toString(){
        String data;
        if (dia < 10) data = "0" + dia;
        else data = "" + dia;
        if (mês < 10) data += "/0" + mês + "/";
        else data += "/" + mês + "/";
        return data += ano;
    }
}
```

Em Python, o conceito de objetos comparáveis suporta a utilização dos comparadores relacionais (`==`, `!=`, `>`, `>=`, `<`, `<=`) para comparar objetos desde que as classes destes objetos implementem os respectivos métodos: `__eq__`, `__ne__`, `__gt__`, `__ge__`, `__lt__` e `__le__`. Em Java essa comparação é feita com a utilização do método `int compareTo(Data data)`. Esse método retorna o valor 1, se a data for superior a data passada como parâmetro, -1 se for inferior e 0 se for igual. Desta forma é possível comparar datas em Java, mas não com a utilização direta dos comparadores relacionais.

Para a implementação do método `calcularIdade` em Java, é utilizada a classe `GregorianCalendar` para obter um objeto com a data atual e constantes da classe `Calendar` para obter dia, mês e ano da data atual. No caso específico de mês, a representação na data atual é no intervalo de 0 a 11, em vez de 1 a 12, de tal forma que é necessário adicionar 1 ao valor correspondente ao mês atual.

A seguir, a implementação da superclasse `Obra` e suas subclasses `Livro` e `TrabalhoPósGraduação` em Python, no módulo `obra` do diretório `entidades`.

```
obras = {}

def get_obras(): return obras

def inserir_obra(obra):
    titulo = obra.titulo
    if titulo not in obras.keys():
        obras[titulo] = obra
    else: print('Erro --- obra já cadastrada com título : ' + titulo)

class Obra:

    def __init__(self, titulo, autor, consulta):
        self.titulo = titulo
        self.autor = autor
        self.consulta = consulta

    def __str__(self):
        obra_str = self.titulo + ' - autor: ' + self.autor
        if self.consulta: obra_str += ' - obra de consulta'
        return obra_str

class Livro(Obra):

    def __init__(self, titulo, autor, consulta, editora, edição, ano_publicação):
        super().__init__(titulo, autor, consulta)
        self.editora = editora
        self.edição = edição
        self.ano_publicação = ano_publicação

    def __str__(self):
        return 'Livro: ' + super().__str__() + '\n editora: ' + self.editora + ' - edição: ' + str(self.edição) + ' - ano de publicação: ' + str(self.ano_publicação)

class TrabalhoPósGraduação(Obra):

    def __init__(self, titulo, autor, consulta, titulação, universidade, orientador, data):
        super().__init__(titulo, autor, consulta)
        self.titulação = titulação if titulação in ('doutorado', 'mestrado', 'especialização')
        else ''
        self.universidade = universidade
        self.orientador = orientador
        self.data = data

    def __str__(self):
        return 'Trabalho de ' + self.titulação + ': ' + super().__str__() + '\n universidade: ' + self.universidade + ' - orientador: ' + self.orientador + ' - data: ' + str(self.data)
```


A seguir, as implementações em Java, das classes [Obra](#), [Livro](#) e [TrabalhoPósGraduação](#) nos seus respectivos arquivos no diretório [entidades](#).

```
package entidades;

import java.util.HashMap;

public class Obra {

    private static HashMap<String, Obra> obras = new HashMap();
    public static HashMap<String, Obra> getObras() { return obras; }

    public static void inserir(Obra obra) {
        String título = obra.getTítulo();
        if (obras.get(título) == null) obras.put(título, obra);
        else System.out.println("Erro --- obra já cadastrada com título : " + título);
    }

    protected String título;
    protected String autor;
    protected boolean consulta;

    public Obra(String título, String autor, boolean consulta) {
        this.título = título;
        this.autor = autor;
        this.consulta = consulta;
    }

    public String getTítulo() { return título; }
    public String getAutor() { return autor; }
    public boolean getConsulta() { return consulta; }

    public String toString () {
        String str = título + " - autor: " + autor;
        if (consulta) str += " - obra de consulta";
        return str;
    }
}

package entidades;

public class Livro extends Obra {

    private String editora;
    private int edição, ano_publicação;

    public Livro(String título, String autor, boolean consulta, String editora, int edição,
        int ano_publicação) {
        super(título, autor, consulta);
        this.editora = editora;
        this.edição = edição;
        this.ano_publicação = ano_publicação;
    }

    public String getEditora() { return editora; }
    public int getEdição() { return edição; }
    public int getAnoPublicação() { return ano_publicação; }

    @Override
    public String toString () {
        return "Livro: " + super.toString()
            + "\n    editora: " + editora
            + " - edição: " + edição
            + " - ano de publicação: " + ano_publicação;
    }
}
```

```
package entidades;

import util.Data;

public class TrabalhoPósGraduação extends Obra {

    public enum Titulação {doutorado, especialização, mestrado}

    private Titulação titulação;
    private String universidade, orientador;
    private Data data;

    public TrabalhoPósGraduação(String título, String autor, boolean consulta, Titulação titulação,
        String universidade, String orientador, Data data) {
        super(título, autor, consulta);
        this.titulação = titulação;
        this.universidade = universidade;
        this.orientador = orientador;
        this.data = data;
    }

    public Titulação getTitulação() { return titulação; }
    public String getUniversidade() { return universidade; }
    public String getOrientador() { return orientador; }
    public Data getData() { return data; }

    @Override
    public String toString () {
        return "Trabalho de " + titulação + ": " + super.toString()
            + "\n    universidade: " + universidade
            + " - orientador: " + orientador
            + " - data: " + data;
    }
}
```

Como foi comentado na seção anterior, cada classe em Java é implementada em um arquivo separado. A superclasse *Obra* não traz nenhum conceito novo, em relação ao que já foi visto na classe *Consulente*.

Nas subclasses existem três diferenças básicas de sintaxe.

A herança é definida como **extends *Obra***, o que denota que as subclasses estendem, com dados e métodos específicos, os dados e métodos herdados da superclasse *Obra*.

De forma equivalente ao construtor em Python, mas com uma sintaxe mais simples, o primeiro comando do construtor de uma subclasse repassa para o construtor da superclasse os argumentos para inicializar os dados herdados: **super(título, autor, consulta);**.

Também de forma equivalente, o método **toString** sobreposto na subclasse, pode chamar o método de mesmo nome herdado da superclasse: **super.toString()**.

3 – Implementando uma entidade de relação

Nesta seção, será ilustrada a classe [Reserva](#) que referencia objetos das classes das seções anteriores: [Consulente](#), [Obra](#) e [Data](#).

A seguir, o código em Python da classe [Reserva](#) definido no módulo [reserva](#) do diretório [entidades](#):

```
from entidades.consulente import get_consulentes
from entidades.obra import get_obras, Livro, TrabalhoPósGraduação

reservas = []

def get_reservas(): return reservas

def inserir_reserva(cpf_consulente, titulo_obra, data):
    consulente = get_consulentes()[cpf_consulente]
    obra = get_obras()[titulo_obra]
    if consulente == None:
        print('Erro -> consulente da reserva não cadastrado')
        return
    if obra == None:
        print('Erro -> obra da reserva não cadastrada')
        return
    reserva = Reserva(consulente, obra, data)
    if reserva not in reservas: reservas.append(reserva)
    else: print('Erro -> reserva já cadastrada com ' + 'cpf de consulente : ' + cpf_consulente
              + ' e título de obra : ' + titulo_obra);

def selecionar_reservas(data_mínima_reserva=None, tipo_consulente=None, obra_consulta=None,
                        ano_publicação_mínimo=None, prefixo_universidade=None):
    filtros = 'Filtros ->'
    if data_mínima_reserva != None:
        filtros += '\n- data mínima reserva: ' + str(data_mínima_reserva)
    if tipo_consulente != None: filtros += ' - tipo do consulente: ' + tipo_consulente
    if obra_consulta: filtros += '\n- não obra de consulta'
    elif obra_consulta == False: filtros += '\n- não obra de consulta'
    if ano_publicação_mínimo != None:
        filtros += ' - ano de publicação mínimo: ' + str(ano_publicação_mínimo)
    if prefixo_universidade != None:
        filtros += ' - prefixo da universidade: ' + prefixo_universidade
    reservas_selecionadas = []
    for reserva in reservas:
        consulente = reserva.consulente
        obra = reserva.obra
        if data_mínima_reserva != None and data_mínima_reserva > reserva.data: continue
        if tipo_consulente != None and consulente.tipo != tipo_consulente: continue
        if obra_consulta in (True, False) and obra.consulta != obra_consulta: continue
        if isinstance(obra, Livro):
            if ano_publicação_mínimo != None and obra.ano_publicação < ano_publicação_mínimo:
                continue
        elif isinstance(obra, TrabalhoPósGraduação):
            if prefixo_universidade != None
              and not obra.universidade.startswith(prefixo_universidade): continue
        reservas_selecionadas.append(reserva)
    return filtros, reservas_selecionadas

class Reserva:

    def __init__(self, consulente, obra, data):
        self.consulente = consulente
        self.obra = obra
        self.data = data

    def __str__(self):
        return 'Reserva para: ' + str(self.data) + '\n      ' + str(self.consulente) + '\n      '
              + str(self.obra)
```

A seguir, o código em Java da classe [Reserva](#) definido no arquivo [Reserva](#) do pacote [entidades](#):

```
package entidades;

import java.util.ArrayList;
import util.Data;
import entidades.Consulente.Tipo;

public class Reserva {

    private static ArrayList<Reserva> reservas = new ArrayList();
    public static ArrayList<Reserva> getReservas() { return reservas; }

    public static void inserir (String cpf_consulente, String titulo_obra, Data data) {
        Consulente consulente = Consulente.getConsulentes().get(cpf_consulente);
        if (consulente == null) {
            System.out.println("Erro -> consulente da reserva não cadastrado");
            return;
        }
        Obra obra = Obra.getObras().get(titulo_obra);
        if (obra == null) {
            System.out.println("Erro -> obra da reserva não cadastrada");
            return;
        }
        Reserva reserva = new Reserva(consulente, obra, data);
        if (!reservas.contains(reserva)) reservas.add(reserva);
        else System.out.println("Erro -> reserva já cadastrada com "
            + "cpf de consulente : " + cpf_consulente + " e título de obra : " + titulo_obra);
    }

    public static ArrayList<Reserva> selecionarReservas(Data data_mínima_reserva,
        Tipo tipo_consulente, char obra_consulta, int menor_ano_publicação,
        String prefixo_universidade) {
        String filtros = "Reservas Selecionadas com Filtros -> ";
        if (data_mínima_reserva != null)
            filtros += "\n- data mínima da reserva: " + data_mínima_reserva;
        if (tipo_consulente != null)
            filtros += " - tipo do consulente: " + tipo_consulente;
        if (obra_consulta == 'T') filtros += "\n- obra de consulta";
        else if (obra_consulta == 'F') filtros += "\n- não obra de consulta";
        if (menor_ano_publicação == -1)
            filtros += " - menor ano de publicação do livro: " + menor_ano_publicação;
        if (prefixo_universidade != null)
            filtros += " - prefixo do nome da universidade da pós: " + prefixo_universidade;
        System.out.println(filtros);
        ArrayList<Reserva> reservas_selecionadas = new ArrayList();
        for (Reserva reserva : reservas) {
            Consulente consulente = reserva.getConsulente();
            Obra obra = reserva.getObra();
            if ((data_mínima_reserva != null)
                && (data_mínima_reserva.compareTo(reserva.getData()) > 0)) continue;
            if ((tipo_consulente != null) && (consulente.getTipo() != tipo_consulente)) continue;
            if (obra_consulta == 'T' && !obra.consulta) continue;
            if (obra_consulta == 'F' && obra.consulta) continue;
            if (obra instanceof Livro) {
                Livro livro = (Livro)obra;
                if ((menor_ano_publicação != -1)
                    && (livro.getAnoPublicação() < menor_ano_publicação)) continue;
            } else if (obra instanceof TrabalhoPósGraduação) {
                TrabalhoPósGraduação tese = (TrabalhoPósGraduação)obra;
                if ((prefixo_universidade != null)
                    && !tese.getUniversidade().startsWith(prefixo_universidade)) continue;
            }
            reservas_selecionadas.add(reserva);
        }
        return reservas_selecionadas;
    }
}
```

```

private Consulente consulente;
private Obra obra;
private Data data;

public Reserva(Consulente consulente, Obra obra, Data data) {
    this.consulente = consulente;
    this.obra = obra;
    this.data = data;
}

public Consulente getConsulente() { return consulente; }
public Obra getObra() { return obra; }
public Data getData() { return data; }

public String toString() {
    return "Reserva para: " + data + "\n - " + consulente + "\n - " + obra;
}
}

```

Em Python podemos armazenar um objeto em uma lista. Em Java podemos obter o mesmo efeito utilizando a classe `ArrayList`, definida a partir do tipo do objeto (`ArrayList<Reserva>`). Como a variável é global e privada deverá ser declarada com `private` e `static`. No exemplo abaixo o primeiro comando declara a variável global `reservas` e inicializa com um objeto da classe `ArrayList`. A criação de um objeto é feita com a utilização da palavra reservada `new` (novo) para chamar o construtor da classe (`ArrayList`). Pelo fato da variável global ser declarada com privada, o seu acesso será realizado a partir do método público e estático (global) `getReservas`, cujo tipo de retorno é o mesmo tipo da variável declarada.

```

private static ArrayList<Reserva> reservas = new ArrayList();
public static ArrayList<Reserva> getReservas() { return reservas; }

```

Na função estática `inserir`, são chamadas as funções estáticas `getConsulentes` e `getObras`. Uma função estática é global e sua chamada é feita a partir do nome da classe. Observe no exemplo seguinte, a função estática `getConsulentes` sendo chamada a partir da classe `Consulente`. Essa chamada vai retornar o objeto `consulentes` da classe `ArrayList`. A partir do objeto `consulentes` retornado é possível obter o `consulente` no `ArrayList` que corresponde à chave `cpf_consulente`, com a chamada do método `get`.

```

Consulente consulente = Consulente.getConsulentes().get(cpf_consulente);

```

Na função `inserir`, também é realizada a criação de um objeto da classe `Reserva`. Para criar um objeto é necessário utilizar a palavra reservada `new` antecedendo o nome do construtor, que tem o mesmo nome de sua classe `Reserva`.

```

Reserva reserva = new Reserva(consulente, obra, data);

```

Na função `inserir` também é ilustrado como verificar se um objeto pertence a uma lista, utilizando o método `contains` da classe `ArrayList`, que retorna um valor do tipo boolean, informando se a lista contém (`true`) ou não (`false`) o objeto. Negação em lugar em Python é representada por `not`, mas em Java é representada pelo caracter `!`:

```

if (!reservas.contains(reserva)) reservas.add(reserva);

```

Em relação à função estática `selecionarReservas`, existem várias observações a serem feitas.

A utilização do filtro `char obra_consulta` que é baseado no atributo `boolean char` da classe `Obra`. O filtro originário de um atributo do tipo `boolean` precisa ter três valores possíveis: verdadeiro, falso ou qualquer (indicando que o filtro não é obrigatório). Um parâmetro do tipo `boolean` só consegue representar dois valores: verdadeiro (`true`) e falso (`false`). Para poder representar também um valor para filtro não obrigatório, para todo filtro originário de um atributo de tipo `boolean` pode ser utilizado o tipo `char` para representar: verdadeiro ('T' indicando `true`), falso ('F' indicando `false`) e qualquer (representado por 'X'). E neste caso o teste se o filtro é atendido é verificado em duas etapas: (a) primeiro com o valor 'T', para o qual a condição deve ser negada para contrariar o valor do filtro, falhar e executar `continue`; e (b) em seguida com o valor 'F', para o qual a condição não é negada para contrariar o valor do filtro, falhar e executar `continue`. Os operadores lógicos em Java são: `&&` para `and`, `||` para `or` e `!` para `not`.

```
if (obra_consulta == 'T' && !obra.consulta) continue;
if (obra_consulta == 'F' && obra.consulta) continue;
```

Para iterar em objetos de uma lista com o comando `for`, é possível utilizar na sua condição a sintaxe `for` (tipo objeto : conjunto). No exemplo seguinte, a cada passada no loop gerado pelo comando `for`, o próximo objeto do conjunto `reservas` é atribuído à variável `reserva` do tipo `Reserva`.

```
for (Reserva reserva : reservas) { ... }
```

As referências e atributos de um objeto são obtidas com a chamada de métodos de leitura:

```
Consulente consulente = reserva.getConsulente(); // obtendo a referência consulente da reserva
Tipo tipo = consulente.getTipo(); // obtendo o atributo tipo do consulente
```

A comparação entre datas, utilizando o método `int compareTo (Data data)` da classe `Data` é ilustrada na seguinte condição, na qual é verificada se o filtro `data_mínima_reserva` tem uma data superior (comparação `> 0`) à data da `reserva`, obtida por `reserva.getData()`.

```
data_mínima_reserva.compareTo(reserva.getData()) > 0)
```

Para utilizar o método `getAnoPublicação`, específico da subclasse `Livro` (não herdado da superclasse `Obra`) a partir da variável `obra` do tipo da superclasse `Obra`, é necessário testar se objeto armazenado na variável `obra` é da subclasse `Livro`, convertê-lo para o objeto `livro` da subclasse `Livro`, para então poder chamar o método específico `getAnoPublicação`.

```
if (obra instanceof Livro) { // testando se o objeto obra é instância da subclasse Livro
    Livro livro = (Livro) obra; // convertendo o objeto obra para um o objeto livro da classe Livro
    if ((menor_ano_publicação != -1)
        && (livro.getAnoPublicação() < menor_ano_publicação)) continue;
```

No método `toString` da classe `Reserva`, observe que não é necessário chamar `consulente.toString()` para converter o objeto para string, dado que o concatenador `+` realiza a chamada do `toString` automaticamente para converter um objeto para string.

4 – Implementando a classe de controle

Nesta seção, será ilustrada a classe `Biblioteca` na qual é realizado o controle das entidades do projeto.

A seguir, o código em Python da classe `Biblioteca` definido no módulo `biblioteca` do diretório `controle`:

```
from util.data import Data
from entidades.consulente import inserir_consulente, Consulente, get_consultentes
from entidades.obra import inserir_obra, Livro, TrabalhoPósGraduação, get_obras
from entidades.reserva import inserir_reserva, get_reservas, selecionar_reservas

def cadastrar_consultentes():
    inserir_consulente(Consulente('Roberta Rodes', '111.111.111-11', 'roberta.rodes@gmail.com',
    'aluno'))
    inserir_consulente(Consulente('Paulo Petri', '222.222.222-22', 'tarso.petri',
    'técnico administrativo'))
    inserir_consulente(Consulente('Renata Riviera', '333.333.333-33', 'renata.riviera@gmail.com',
    'professor'))

def cadastrar_obras():
    inserir_obra(Livro(título='Desenvolvimento de Aplicações com Python', autor='Felipe Andrade',
    consulta=False, editora='Técnica', edição=3, ano_publicação=2017))
    inserir_obra(Livro('Desenvolvimento de Aplicações com Java', 'Felipe Andrade', False, 'Técnica',
    2, 2010))
    inserir_obra(Livro('Desenvolvimento Web com Java', 'Silvia Saraiva', False, 'Publicadora', 2,
    2015))
    inserir_obra(TrabalhoPósGraduação(título='Desenvolvimento de Aplicações Críticas',
    autor='Paulo Machado', consulta=False, titulação='doutorado', universidade='Unicamp',
    orientador='Roberto Sobral', data=Data(29, 3, 2007)))
    inserir_obra(TrabalhoPósGraduação('Deep Learning em Processamento de Linguagem Natural',
    'Ariana Silveira', False, 'mestrado', 'UFMS', 'Fábio Pedroso', Data(29, 3, 2007)))
    inserir_obra(TrabalhoPósGraduação('Desenvolvimento de Aplicações em Tempo Real',
    'Tatiana Dunski', True, 'especialização', 'UFRGS', 'Camila Baun', Data(29, 3, 2007)))

def cadastrar_reservas():
    inserir_reserva('111.111.111-11', 'Desenvolvimento de Aplicações com Python', Data(1, 4, 2022))
    inserir_reserva('111.111.111-11', 'Deep Learning em Processamento de Linguagem Natural',
    Data(10, 4, 2022))
    inserir_reserva('111.111.111-11', 'Desenvolvimento de Aplicações com Java', Data(10, 4, 2022))
    inserir_reserva('111.111.111-11', 'Desenvolvimento de Aplicações Críticas', Data(10, 4, 2022))
    inserir_reserva('111.111.111-11', 'Desenvolvimento Web com Java', Data(21, 4, 2022))
    inserir_reserva('111.111.111-11', 'Desenvolvimento de Aplicações em Tempo Real',
    Data(21, 4, 2022))
    inserir_reserva('222.222.222-22', 'Desenvolvimento de Aplicações com Java', Data(15, 4, 2022))
    inserir_reserva('333.333.333-33', 'Desenvolvimento de Aplicações em Tempo Real',
    Data(18, 4, 2022))

def imprimir_objetos(cabeçalho, objetos, filtros=None):
    if filtros == None: print('\n' + cabeçalho)
    else: print ('\n' + cabeçalho + filtros)
    for índice, objeto in enumerate(objetos): print(str(índice + 1) + ' - ' + str(objeto))
```

```
if __name__ == '__main__':
    cadastrar_consultentes()
    imprimir_objetos('Consultentes Cadastrados', get_consultentes().values())
    cadastrar_obras()
    imprimir_objetos('Obras Cadastradas', get_obras().values())
    cadastrar_reservas()
    imprimir_objetos('Reservas Cadastradas', get_reservas())
    cabeçalho_reservas_selecionadas = 'Reservas Selecionadas com '
    filtros, reservas_selecionadas = selecionar_reservas(data_mínima_reserva=Data(10,4,2022))
    imprimir_objetos(cabeçalho_reservas_selecionadas, reservas_selecionadas, filtros)
    filtros, reservas_selecionadas = selecionar_reservas(Data(10,4,2022), tipo_consultente='aluno')
    imprimir_objetos(cabeçalho_reservas_selecionadas, reservas_selecionadas, filtros)
    filtros, reservas_selecionadas = selecionar_reservas(Data(10,4,2022), 'aluno',
        obra_consulta=False)
    imprimir_objetos(cabeçalho_reservas_selecionadas, reservas_selecionadas, filtros)
    filtros, reservas_selecionadas = selecionar_reservas(Data(10,4,2022), 'aluno', False,
        ano_publicação_mínimo=2013)
    imprimir_objetos(cabeçalho_reservas_selecionadas, reservas_selecionadas, filtros)
    filtros, reservas_selecionadas = selecionar_reservas(Data(10,4,2022), 'aluno', False, 2013,
        prefixo_universidade='UF')
    imprimir_objetos(cabeçalho_reservas_selecionadas, reservas_selecionadas, filtros)
```

A seguir, o código em Java da classe [Biblioteca](#) definido no arquivo [Biblioteca](#) do pacote [controle](#):

```
package controle;

import util.Data;
import entidades.Consultente;
import entidades.Consultente.Tipo;
import entidades.Obra;
import entidades.Livro;
import entidades.TrabalhoPósGraduação;
import entidades.TrabalhoPósGraduação.Titulação;
import entidades.Reserva;

public class Biblioteca {

    private static void cadastrarConsultentes() {
        Consultente.inserir(new Consultente("Roberta Rodes", "111.111.111-11",
            "roberta.rodes@gmail.com", Tipo.aluno));
        Consultente.inserir(new Consultente("Paulo Petri", "222.222.222-22", "tarso.petri",
            Tipo.técnico_administrativo));
        Consultente.inserir(new Consultente("Renata Riviera", "333.333.333-33",
            "renata.riviera@gmail.com", Tipo.professor));
    }

    private static void cadastrarObras() {
        Obra.inserir(new Livro("Desenvolvimento de Aplicações com Python", "Felipe Andrade", false,
            "Técnica", 3, 2017));
        Obra.inserir(new Livro("Desenvolvimento de Aplicações com Java", "Felipe Andrade", false,
            "Técnica", 2, 2010));
        Obra.inserir(new Livro("Desenvolvimento Web com Java", "Silvia Saraiva", false,
            "Publicadora", 2, 2015));
        Obra.inserir(new TrabalhoPósGraduação("Desenvolvimento de Aplicações Críticas",
            "Paulo Machado", false, Titulação.doutorado, "Unicamp", "Roberto Sobral",
            new Data(29, 3, 2007));
        Obra.inserir(new TrabalhoPósGraduação("Deep Learning em Processamento de Linguagem Natural",
            "Ariana Silveira", false, Titulação.mestrado, "UFMS", "Fábio Pedroso",
            new Data(29, 3, 2007));
        Obra.inserir(new TrabalhoPósGraduação("Desenvolvimento de Aplicações em Tempo Real",
            "Tatiana Dunski", true, Titulação.especialização, "UFRGS", "Camila Baun",
            new Data(29, 3, 2007));
    }
}
```



```
private static void cadastrarReservas() {
    Reserva.inserir("111.111.111-11", "Desenvolvimento de Aplicações com Python",
        new Data(1,4,2022));
    Reserva.inserir("111.111.111-11", "Deep Learning em Processamento de Linguagem Natural",
        new Data(10,4,2022));
    Reserva.inserir("111.111.111-11", "Desenvolvimento de Aplicações com Java",
        new Data(10,4,2022));
    Reserva.inserir("111.111.111-11", "Desenvolvimento de Aplicações Críticas",
        new Data(10,4,2022));
    Reserva.inserir("111.111.111-11", "Desenvolvimento Web com Java", new Data(21,4,2022));
    Reserva.inserir("111.111.111-11", "Desenvolvimento de Aplicações em Tempo Real",
        new Data(21,4,2022));
    Reserva.inserir("222.222.222-22", "Desenvolvimento de Aplicações com Java",
        new Data(15,4,2022));
    Reserva.inserir("333.333.333-33", "Desenvolvimento de Aplicações em Tempo Real",
        new Data(18,4,2022));
}

private static void imprimirObjetos(String cabeçalho, Iterable objetos) {
    if (cabeçalho != null) System.out.println(cabeçalho);
    int n = 1;
    for (Object objeto : objetos) System.out.println(n++ + " - " + objeto);
    System.out.println();
}

public static void main(String[] args) {
    cadastrarConsulentes();
    cadastrarObras();
    cadastrarReservas();
    imprimirObjetos("Consulentes Cadastrados", Consulente.getConsulentes().values());
    imprimirObjetos("Obras Cadastradas", Obra.getObras().values());
    imprimirObjetos("Reservas Cadastradas", Reserva.getReservas());
    imprimirObjetos(null, Reserva.selecionarReservas(new Data(10,4,2022), null, 'X', -1, null));
    imprimirObjetos(null, Reserva.selecionarReservas(new Data(10,4,2022), Tipo.aluno, 'X',
        -1, null));
    imprimirObjetos(null, Reserva.selecionarReservas(new Data(10,4,2022), Tipo.aluno, 'F',
        -1, null));
    imprimirObjetos(null, Reserva.selecionarReservas(new Data(10,4,2022), Tipo.aluno, 'F',
        2013, null));
    imprimirObjetos(null, Reserva.selecionarReservas(new Data(10,4,2022), Tipo.aluno, 'F',
        2013, "UF"));
}
```

Para passar um elemento do enumerado **Tipo** como parâmetro é necessário utilizar o nome do tipo e o nome do elemento. Para utilizar o tipo **Tipo** da classe **Consulente** na classe **Biblioteca** existem duas alternativas: (a) importar **Consulente.Tipo** e utilizar **Tipo.aluno**; ou (b) importar somente a classe **Consulente** e utilizar **Consulente.Tipo.aluno**. Se forem importados tipos enumerados com o mesmo nome de duas classes distintas, então será necessário utilizar a segunda alternativa, de forma que o nome da classe seja utilizado para distinguir os tipos de mesmo nome. Neste tutorial, está sendo utilizada a primeira alternativa, como ilustrado no trecho de código abaixo:

```
import entidades.Consulente.Tipo;
public class Biblioteca {
    private static void cadastrarConsulentes() {
        Consulente.inserir(new Consulente("Roberta Rodes", "111.111.111-11",
            "roberta.rodes@gmail.com", Tipo.aluno));
        Consulente.inserir(new Consulente("Paulo Petri", "222.222.222-22", "tarso.petri",
            Tipo.técnico_administrativo));
        Consulente.inserir(new Consulente("Renata Riviera", "333.333.333-33",
            "renata.riviera@gmail.com", Tipo.professor));
    }
}
```

Lembre-se que a chamada dos construtores dos objetos é sempre precedida pela palavra reservada **new** (novo), indicando a criação de um novo objeto.

```
Consulente.inserir(new Consulente("Roberta Rodes", "111.111.111-11",
    "roberta.rodes@gmail.com", Tipo.aluno));
```

A palavra reservada para verdadeiro em Python é **True** e em Java é **true**. De forma semelhante a palavra reservada para falso em Python é **False** e em Java é **false**.

```
Obra.inserir(new TrabalhoPósGraduação(
    "Deep Learning em Processamento de Linguagem Natural", "Ariana Silveira",
    false, Titulação.mestrado, "UFMS", "Fábio Pedroso", new Data(29, 3, 2007)));
Obra.inserir(new TrabalhoPósGraduação("Desenvolvimento de Aplicações em Tempo Real",
    "Tatiana Dunski", true, Titulação.especialização, "UFRGS", "Camila Baun",
    new Data(29, 3, 2007)));
```

A função estática **imprimirObjetos** recebe como argumentos: cabeçalho e conjunto de objetos. O segundo argumento é do tipo **Iterable**, o que lhe permite receber vários tipos de conjuntos iteráveis.

```
private static void imprimirObjetos(String cabeçalho, Iterable objetos) {
```

A função estática **imprimirObjetos** é chamada para imprimir conjuntos de objetos que retornam diferentes tipos. No entanto, apesar de diferentes todos esses tipos são iteráveis, o que pode ser resolvido com a definição de um argumento do tipo **Iterable**.

```
imprimirObjetos("Consulentes Cadastrados", Consulente.getConsulentes().values());
imprimirObjetos("Obras Cadastradas", Obra.getObras().values());
imprimirObjetos("Reservas Cadastradas", Reserva.getReservas());
imprimirObjetos(null, Reserva.selecionarReservas(new Data(10,4,2022), null, 'X', -1, null));
```

O conjunto **consulentes** (definido na classe **Consulente**) do tipo **HashMap<String, Consulente>** armazena objetos da classe **Consulente** associados a chaves do tipo **String**. Idem para o conjunto **obras** do tipo **HashMap<String, Obra>**. O método **values** é utilizado, ao chamar a função **imprimirObjetos** para passar como argumento somente um conjunto de objetos, e não os pares chave e objeto.

5 – A saída dos projetos em Python e em Java

Obviamente, para fazer sentido a comparação entre o código Python e o código Java, essas duas versões do projeto **Biblioteca** foram implementadas para gerar a mesma saída. Neste seção, é ilustrada a saída gerada por essas duas versões do projeto. Observe que os filtros associados a subclasses só causam restrições para objetos da respectiva subclasse.

```
Consulentes Cadastrados
1 - Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
2 - Consulente (técnico administrativo): Paulo Petri - CPF: 222.222.222-22 - email: tarso.petri
3 - Consulente (professor): Renata Riviera - CPF: 333.333.333-33 - email: renata.riviera@gmail.com
```

Linguagem de Programação II - Tutorial Auxiliar - 19/21

Obras Cadastradas

- 1 - Livro: Desenvolvimento de Aplicações com Python - autor:Felipe Andrade
editora: Técnica - edição: 3 - ano de publicação: 2017
- 2 - Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade
editora: Técnica - edição: 2 - ano de publicação: 2010
- 3 - Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva
editora: Publicadora - edição: 2 - ano de publicação: 2015
- 4 - Trabalho de doutorado: Desenvolvimento de Aplicações Críticas - autor:Paulo Machado
universidade: Unicamp - orientador: Roberto Sobral - data: 29/03/2007
- 5 - Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural
- autor:Ariana Silveira
universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007
- 6 - Trabalho de especialização: Desenvolvimento de Aplicações em Tempo Real - autor:Tatiana Dunski
- obra de consulta
universidade: UFRGS - orientador: Camila Baun - data: 29/03/2007

Reservas Cadastradas

- 1 - Reserva para: 01/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento de Aplicações com Python - autor:Felipe Andrade
editora: Técnica - edição: 3 - ano de publicação: 2017
- 2 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural
- autor:Ariana Silveira
universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007
- 3 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade
editora: Técnica - edição: 2 - ano de publicação: 2010
- 4 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de doutorado: Desenvolvimento de Aplicações Críticas - autor:Paulo Machado
universidade: Unicamp - orientador: Roberto Sobral - data: 29/03/2007
- 5 - Reserva para: 21/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva
editora: Publicadora - edição: 2 - ano de publicação: 2015
- 6 - Reserva para: 21/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de especialização: Desenvolvimento de Aplicações em Tempo Real - autor:Tatiana Dunski
- obra de consulta
universidade: UFRGS - orientador: Camila Baun - data: 29/03/2007
- 7 - Reserva para: 15/04/2022
Consulente (técnico administrativo): Paulo Petri - CPF: 222.222.222-22 - email: tarso.petri
Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade
editora: Técnica - edição: 2 - ano de publicação: 2010
- 8 - Reserva para: 18/04/2022
Consulente (professor): Renata Riviera - CPF: 333.333.333-33 - email: renata.riviera@gmail.com
Trabalho de especialização: Desenvolvimento de Aplicações em Tempo Real - autor:Tatiana Dunski
- obra de consulta
universidade: UFRGS - orientador: Camila Baun - data: 29/03/2007

Linguagem de Programação II - Tutorial Auxiliar - 20/21

Reservas Selecionadas com Filtros ->

- data mínima reserva: 10/04/2022

- 1 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural
- autor:Ariana Silveira
universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007
- 2 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade
editora: Técnica - edição: 2 - ano de publicação: 2010
- 3 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de doutorado: Desenvolvimento de Aplicações Críticas - autor:Paulo Machado
universidade: Unicamp - orientador: Roberto Sobral - data: 29/03/2007
- 4 - Reserva para: 21/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva
editora: Publicadora - edição: 2 - ano de publicação: 2015
- 5 - Reserva para: 21/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de especialização: Desenvolvimento de Aplicações em Tempo Real - autor:Tatiana Dunski
- obra de consulta
universidade: UFRGS - orientador: Camila Baun - data: 29/03/2007
- 6 - Reserva para: 15/04/2022
Consulente (técnico administrativo): Paulo Petri - CPF: 222.222.222-22 - email: tarso.petri
Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade
editora: Técnica - edição: 2 - ano de publicação: 2010
- 7 - Reserva para: 18/04/2022
Consulente (professor): Renata Riviera - CPF: 333.333.333-33 - email: renata.riviera@gmail.com
Trabalho de especialização: Desenvolvimento de Aplicações em Tempo Real - autor:Tatiana Dunski
- obra de consulta
universidade: UFRGS - orientador: Camila Baun - data: 29/03/2007

Reservas Selecionadas com Filtros ->

- data mínima reserva: 10/04/2022 - tipo do consulente: aluno

- 1 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural
- autor:Ariana Silveira
universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007
- 2 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade
editora: Técnica - edição: 2 - ano de publicação: 2010
- 3 - Reserva para: 10/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de doutorado: Desenvolvimento de Aplicações Críticas - autor:Paulo Machado
universidade: Unicamp - orientador: Roberto Sobral - data: 29/03/2007
- 4 - Reserva para: 21/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva
editora: Publicadora - edição: 2 - ano de publicação: 2015
- 5 - Reserva para: 21/04/2022
Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com
Trabalho de especialização: Desenvolvimento de Aplicações em Tempo Real - autor:Tatiana Dunski
- obra de consulta
universidade: UFRGS - orientador: Camila Baun - data: 29/03/2007

Linguagem de Programação II - Tutorial Auxiliar - 21/21

Reservas Selecionadas com Filtros ->

- data mínima reserva: 10/04/2022 - tipo do consulente: aluno

- não obra de consulta

1 - Reserva para: 10/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural

- autor:Ariana Silveira

universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007

2 - Reserva para: 10/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Livro: Desenvolvimento de Aplicações com Java - autor:Felipe Andrade

editora: Técnica - edição: 2 - ano de publicação: 2010

3 - Reserva para: 10/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Trabalho de doutorado: Desenvolvimento de Aplicações Críticas - autor:Paulo Machado

universidade: Unicamp - orientador: Roberto Sobral - data: 29/03/2007

4 - Reserva para: 21/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva

editora: Publicadora - edição: 2 - ano de publicação: 2015

Reservas Selecionadas com Filtros ->

- data mínima reserva: 10/04/2022 - tipo do consulente: aluno

- não obra de consulta - ano de publicação mínimo: 2013

1 - Reserva para: 10/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural

- autor:Ariana Silveira

universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007

2 - Reserva para: 10/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Trabalho de doutorado: Desenvolvimento de Aplicações Críticas - autor:Paulo Machado

universidade: Unicamp - orientador: Roberto Sobral - data: 29/03/2007

3 - Reserva para: 21/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva

editora: Publicadora - edição: 2 - ano de publicação: 2015

Reservas Selecionadas com Filtros ->

- data mínima reserva: 10/04/2022 - tipo do consulente: aluno

- não obra de consulta - ano de publicação mínimo: 2013 - prefixo da universidade: UF

1 - Reserva para: 10/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Trabalho de mestrado: Deep Learning em Processamento de Linguagem Natural

- autor:Ariana Silveira

universidade: UFMS - orientador: Fábio Pedroso - data: 29/03/2007

2 - Reserva para: 21/04/2022

Consulente (aluno): Roberta Rodes - CPF: 111.111.111-11 - email: roberta.rodes@gmail.com

Livro: Desenvolvimento Web com Java - autor:Silvia Saraiva

editora: Publicadora - edição: 2 - ano de publicação: 2015